



JÖNKÖPING UNIVERSITY
School of Engineering

Labbrapport

Labbpaket 1

Förnamn Efternamn, Förnamn Efternamn, Förnamn Efternamn



v. 1.0

Sammanfattning: Allt för ofta plågas studenter med Word eller andra ordhanterare som inte kan hantera ekvationer, figurer eller tabeller på ett tillförlitligt och tillfredsställande sätt. Denna mall visar hur man med Quarto och Jupyter Notebooks kan generera snygga PDF-rapporter via LaTeX. Innehållet hamnar i fokus, inte formateringen. Studera detta dokument och dess celler för att lära er hur det är uppbyggt. Håll rapporten snygg, kort och koncis!

1 Introduktion - Hur man skriver moderna rapporter

Denna mall använder [Quarto](#) med Jupyter Notebooks för att generera PDF-rapporter via LaTeX. Till skillnad från Overleaf skriver vi i vanliga Jupyter Notebooks och använder LaTeX *enbart* för matematiska ekvationer. Quarto hanterar all formatering, numrering av sektioner, ekvationer och figurer samt referenshantering automatiskt.

1.1 Installation

För att kunna rendera denna mall behövs:

1. **Python och Jupyter** installerat, se [installationsguiden](#)
2. **Quarto** installerat från [quarto.org](#). Välj “Install for me only” om du saknar administratörsrättigheter.
3. **En LaTeX-distribution** för PDF-rendering. Kontrollera om du redan har en med `pdflatex --version`. Om inte:
 - Linux: `sudo apt install texlive-full`
 - macOS: [MacTeX](#)
 - Windows: [MiKTeX](#)
 - Alternativt TinyTeX (liten, ~250 MB): `quarto install tinytex`
4. **VS Code** med tilläggen [Quarto](#) och [Jupyter](#)

Se [PDF Reports with LaTeX](#) för en fullständig genomgång av Quarto, LaTeX-matematik och felsökning.

1.2 Arbetsflöde

1. Öppna denna notebook i VS Code
2. Redigera text i markdown-celler, lägg till ekvationer med LaTeX-syntax
3. Lägg till Python-kodceller för figurer och beräkningar
4. Rendera till PDF med ett av följande kommandon:

```
quarto preview report.ipynb --to pdf
```

`quarto preview` renderar PDF:en och öppnar den automatiskt. Dessutom bevakar den filen, så varje gång du sparar notebooken renderas PDF:en om och uppdateras. Perfekt under skrivandet, precis som Overleafs live-förhandsgranskning. Avsluta med **Ctrl+C** i terminalen.

```
quarto render report.ipynb --to pdf --execute
```

`quarto render` renderar PDF:en en gång utan bevakning. Flaggan `--execute` kör alla kodceller vid renderingen, vilket säkerställer reproducerbara resultat. Använd detta för slutversionen.

1.3 Dokumentstruktur

Hela rapporten finns i en enda notebook-fil. Metadata (titel, författare, format) definieras i YAML-blocket i den första cellen (en *raw*-cell). Markdown-celler innehåller text och ekvationer, och kodceller genererar figurer och tabeller automatiskt.

2 Figurer

Se Figur 1, en graf av högsta kvalitet.



Figur 1: Look at this graph

2.1 Hur hanterar vi figurer?

Titta på källkoden i notebook-cellerna för att se hur man inkluderar en figur med figurtext och referens. Syntaxen är:

```
! [Figurtext] (sökväg/till/bild.jpg) {#fig-etikett width=60%}
```

Referera sedan i texten med @fig-etikett. Quarto numrerar figurer automatiskt.

Vi tar alltid med figurtext och beskriver figurerna väl. Är det inte egenskapade figurer bör vi hänvisa till källan.

2.2 Vektoriserad grafik

Sträva alltid efter att använda vektoriserade bilder (SVG, PDF) som kan skalas godtyckligt utan kvalitetsförlust. Man kan använda PowerPoint eller [Inkscape](#) för att skapa grafik och exportera som SVG.

2.3 Figurer från Python-kod

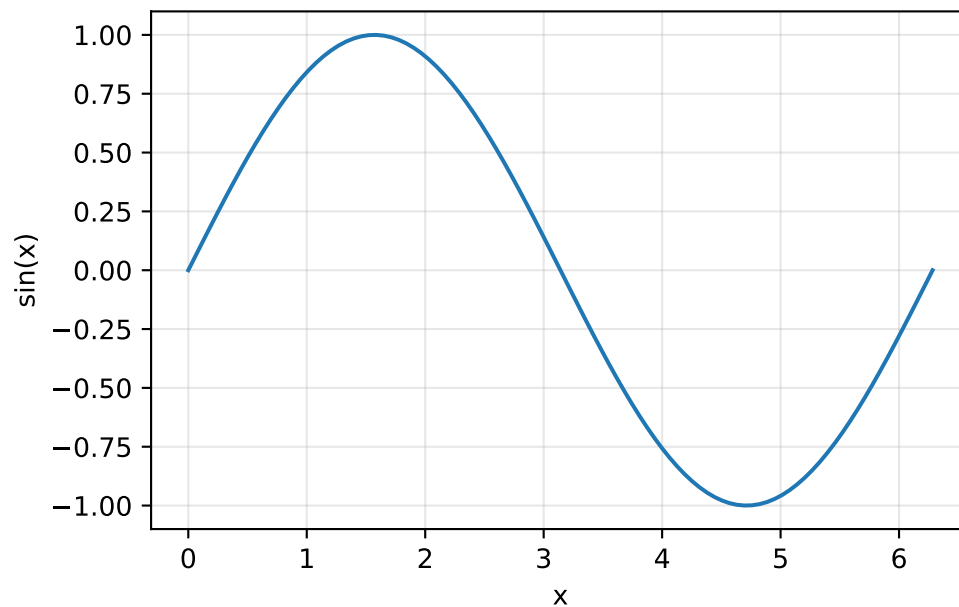
En av Quartos stora styrkor är att Python-kod kan generera figurer direkt i rapporten. Nedanstående kodcell genererar Figur 2 med följande Quarto-kommentarer:

```
#| label: fig-sinuswave  
#| fig-cap: "En sinusvåg genererad med Python"  
#| echo: true
```

- label: fig-sinuswave gör figuren referensbar med @fig-sinuswave
- fig-cap sätter figurtexten
- echo: true visar koden i PDF:en (byt till false för att dölja den)

```
import numpy as np  
import matplotlib.pyplot as plt
```

```
x = np.linspace(0, 2*np.pi, 100)
plt.plot(x, np.sin(x))
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.grid(True, alpha=0.3)
plt.show()
```



Figur 2: En sinusvåg genererad med Python

Kodcellen ovan använder Quarto-kommentarer (`#|`) för att kontrollera utseendet, se Tabell 1.

Tabell 1: Quarto-kommentarer för kodceller

Alternativ	Effekt
<code># echo: true</code>	Visa koden (standard)
<code># echo: false</code>	Dölj koden, visa bara figuren
<code># output: false</code>	Dölj utdata
<code># warning: false</code>	Undertryck varningar
<code># label: fig-namn</code>	Skapa referensbar figur
<code># fig-cap: "..."</code>	Figurtext

3 Matematisk notation

“En ekvation säger mer än tusen figurer”

— Fredrik Larsson

Det är viktigt att en ingenjör är precis i sitt språk för att vara effektiv i kommunikationen och undvika missförstånd. Meningen är att vara precisa i språket för att undvika missförstånd och få mottagaren att kunna följa samt återskapa arbetet. Nedan kan vi beskåda typisk matematisk notation som en maskiningenjör utsätter sin omgivning för.

3.1 Ekvationer i text och på egen rad

Använd enkla dollartecken $\$ \dots \$$ för matematik i löpande text och dubbla $\$\$ \dots \$\$$ för centrerade ekvationer.

Modellen bygger på att kinetisk energi omvandlas till potentiell energi samt lite kinetisk energi, tåget ska alltså ha en initial hastighet v_0 som ska ta upp tåget till höjden h där tåget ska ha en hastighet v .

$$\frac{1}{2}mv_0^2 = mgh + \frac{1}{2}mv^2$$

Vi löser ut v

$$v = \sqrt{v_0^2 - 2gh} \tag{1}$$

I positiv normalriktning får vi

$$N - mg \cos \theta = ma_c$$

Normalaccelerationen i en kurvad bana ges av

$$a_c = \frac{v^2}{r}$$

Insättning ger

$$N = \frac{mv^2}{r} + mg \cos \theta$$

vi dividerar bort massan samt g för att få uttrycket i antal g -“krafter”

$$\frac{N}{mg} = \frac{v^2}{rg} + \cos \theta =: G$$

Konstant normal-acceleration medför

$$a_c = Cg = \frac{v^2}{r} \quad (2)$$

Sätt Ekvation 1 i Ekvation 2

$$a_c = Cg = \frac{v_0^2 - 2gy}{Cg}$$

$$\text{låt } h=y \Rightarrow \boxed{r = \frac{v_0^2 - 2gh}{Cg}}$$

Förhållandet mellan bansträcka, radie och vinkel ges av

$$ds = r d\theta \Rightarrow \frac{d\theta}{ds} = \frac{1}{r}$$

vilket leder till en slutlig trevlig ordinär differentialekvation som löses numeriskt med Python.

$$\text{ODE} \begin{cases} \frac{d\theta}{ds} = \frac{1}{r} \\ \frac{dx}{ds} = \cos \theta \\ \frac{dy}{ds} = \sin \theta \end{cases}$$

3.2 Vektorer och matriser

- Vektorer skrivs vanligtvis fetstilt: $\mathbf{a}$ och $\boldsymbol{\varphi}$, eller med \bm: $\boldsymbol{\sigma}$.
- Kursiv stil för tensorer: $\mathcal{T}, \mathcal{C}_{ijkl}$. En matris kan skrivas $\mathbf{A}$ och den numeriska versionen $\mathbf{A}$.
- Det finns massor av olika sätt att skriva transponat. Den lata: $\mathbf{a}^T$, den mindre lata $\mathbf{a}^\top$, den kanske snyggare $\mathbf{a}^\top$.
- Vi skriver kraftvektorer med $\mathbf{F}$ eller $\mathbf{F}$

Basfunktionsmatrisen definieras enligt,

$$\boldsymbol{\Phi} := \begin{bmatrix} \varphi_1 & 0 & 0 & \varphi_2 & 0 & 0 & \cdots \\ 0 & \varphi_1 & 0 & 0 & \varphi_2 & 0 & \cdots \\ 0 & 0 & \varphi_1 & 0 & 0 & \varphi_2 & \cdots \end{bmatrix} \quad (3)$$

3.3 Stora matriser

För matriser med bråk behöver vi ibland justera radavståndet. Här använder vi rå LaTeX:

$$\mathbf{J}(\xi, \eta, \zeta) = \begin{bmatrix} \frac{\partial x_h}{\partial \xi} & \frac{\partial y_h}{\partial \xi} & \frac{\partial z_h}{\partial \xi} \\ \frac{\partial x_h}{\partial \eta} & \frac{\partial y_h}{\partial \eta} & \frac{\partial z_h}{\partial \eta} \\ \frac{\partial x_h}{\partial \zeta} & \frac{\partial y_h}{\partial \zeta} & \frac{\partial z_h}{\partial \zeta} \end{bmatrix}$$

3.4 Numerisk evaluering

Ibland måste vi skriva ut en uträkning eller evaluering av en funktion:

$$\frac{dy}{dt} = \dot{y} = \frac{125\pi}{200} \sin\left(\frac{50\pi}{200}\right) \cdot 150 = \frac{375\pi\sqrt{2}}{8} \approx 208 \text{ m/s}$$

För varje felaktig användning av symbolen $*$ för gångertecken gråter en matematiker eller ingenjör lite inombords.

$$f * g$$

betyder faltning¹ av f med g .

3.5 Gaussisk integration

$$\int_0^1 f(x) dx \approx \sum_{i=1}^n f(x_i) w_i$$

En n -punkts Gaussisk kvadrering ger en exakt lösning för polynom av graden $2n - 1$ eller lägre. Se Tabell 2.

Tabell 2: Gaussisk kvadratur för $\xi \in [0, 1]$

Gauss-ordning n	Polynomgrad	Vikter w_i	Punkter x_i
1	1	1	$\frac{1}{2}$
2	3	$\frac{1}{2}, \frac{1}{2}$	$\frac{1}{2} \pm \frac{\sqrt{3}}{6}$
3	5	$\frac{5}{18}, \frac{4}{9}, \frac{5}{18}$	Se nedan

Gauss-punkter för $n = 3$: $\frac{1}{2} - \frac{\sqrt{15}}{10}, \frac{1}{2}, \frac{1}{2} + \frac{\sqrt{15}}{10}$.

För godtyckliga Gaussgrader och spann kan man använda Golub-Welsh-algoritmen.

4 Tabeller

4.1 Enkla tabeller i markdown

Kolumn 1	Kolumn 2	Kolumn 3
Data	Data	Data
Data	Data	Data

4.2 Komplexa tabeller med LaTeX

För tabeller med ekvationer eller speciell formatering kan man använda rå LaTeX. Med `\resizebox{\textwidth}{!}{...}` skalar vi tabellen till textbredden:

¹<https://en.wikipedia.org/wiki/Convolution>

State of stress	Boundary Conditions	von Mises Equations
General 3D	No restrictions	$\sigma_v = \sqrt{\frac{1}{2} [(\sigma_x - \sigma_y)^2 + (\sigma_y - \sigma_z)^2 + (\sigma_z - \sigma_x)^2 + 6(\sigma_{xy}^2 + \sigma_{yz}^2 + \sigma_{zx}^2)]}$
General plane stress (2D)	$\sigma_z = \sigma_{yz} = \sigma_{zx} = 0$	$\sigma_v = \sqrt{\sigma_x^2 - \sigma_x \sigma_y + \sigma_y^2 + 3\sigma_{xy}^2}$

5 Inkludera kod i rapporter

I Quarto visas Python-kod direkt från kodceller. Kontrollera visningen med `#|`-kommentarer i toppen av kodcellen:

- `#| echo: true` – visa koden (standard)
- `#| echo: false` – dölj koden, visa bara resultat
- `#| output: false` – dölj resultat

Nedan visas ett kodexempel med `#| echo: true`:

```
import numpy as np

# Elementloop-exempel
n_elements = 4
n_nodes = n_elements + 1
S = np.zeros((n_nodes, n_nodes))
F = np.zeros(n_nodes)

for iel in range(n_elements):
    inod = [iel, iel + 1]      # Element nodes
    Sk = np.array([[1, -1],   # Element stiffness
                  [-1, 1]])
    fk = np.array([0.5, 0.5]) # Element force

    # Assembly
    ix = np.ix_(inod, inod)
    S[ix] += Sk
    F[inod] += fk

print("Stiffness matrix:")
print(S)
print("Force vector:", F)
```

```
Stiffness matrix:
[[ 1. -1.  0.  0.  0.]
 [-1.  2. -1.  0.  0.]
 [ 0. -1.  2. -1.  0.]
 [ 0.  0. -1.  2. -1.]
 [ 0.  0.  0. -1.  1.]]
Force vector: [0.5 1.  1.  1.  0.5]
```

6 Referenshantering

Quarto hanterar referenser via `.bib`-filer. I YAML-blocket anger vi:


```
bibliography: library.bib
```

Citera sedan med @nyckel eller [nyckel]:

- @Stromberg2012 ger: N. Strömberg (2012)
- [@Johannesson2013; @Logan2012] ger: (H. Johannesson, J-G Persson and D Pettersson 2013; D. L. Logan 2012)

Ibland behöver man referera till samma smarta människor som har gjort bra grejer, se (H. Johannesson, J-G Persson and D Pettersson 2013; D. L. Logan 2012; B. Lundkvist 1997; N. Strömberg 2012).

6.1 Referensstil

Referensstilen ändras med en CSL-fil i YAML. Exempelvis för IEEE-stil:

```
csl: ieee.csl
```

Ladda ner `ieee.csl` från [Zotero Style Repository](#) och lägg filen i samma katalog som notebooken. Vanliga stilar:

Stil	Filnamn	Format
IEEE	<code>ieee.csl</code>	[1], [2], [3]
APA	<code>apa.csl</code>	(Strömberg, 2012)
Vancouver	<code>vancouver.csl</code>	(1), (2), (3)

6.2 BibTeX-format

Referensdata lagras i `library.bib` i BibTeX-format:

```
@Book{Callister2011,  
  author = {{W. D Callister and D. G Rethwisch}},  
  title  = {Materials Science and Engineering},  
  year   = {2011},  
  volume = {SI version},  
  edition = {8},  
}
```

Man kan söka “titel + author bibtex” eller använda Google Scholars cite-knapp med BibTeX-länken.

Alla citerade referenser läggs automatiskt i slutet av dokumentet.

Referenser

B. Lundkvist. 1997. *Ritteknik*. 7:e uppl. Liber.

D. L. Logan. 2012. *A first course in the finite element method*. Fifth. Global Engineering.

H. Johannesson, J-G Persson and D Pettersson. 2013. *Produktutveckling: Effektiva metoder för konstruktion och design*.

N. Strömberg. 2012. *Nonlinear FEA and Design Optimization for Mechanical Engineers*. 3:e uppl.